Certified RDE Certification — Hands-On Project

Smart Backlog Assistant: Plain-English Guide

Problem Definition • Solution Design • Prompt Strategy • Code Review Guide • How to Initialize & Run

IMPORTANT CODE REVIEW NOTE:
• **Inspect the agent/ folder only:** To evaluate the AI Agent, Prompts, Subagents, MCPs, Hooks, and Tools, reviewers only need to check the dedicated **agent/** directory. All AI components are cleanly condensed there without UI clutter.
• **Do NOT delete src/backlog_assistant/:** The code in src/backlog_assistant/ is the active, runnable package that powers the Streamlit web portal (app.py), terminal CLI, and automated pytest test suite. It is required for live execution.

1. What Problem Does This Solve? (Step 1)

After every planning meeting, outage, or compliance review, engineers end up with messy notes, Markdown files, or PDFs. Turning those notes into clean Jira tickets by hand takes hours—and asking a basic AI to do it usually causes four headaches:

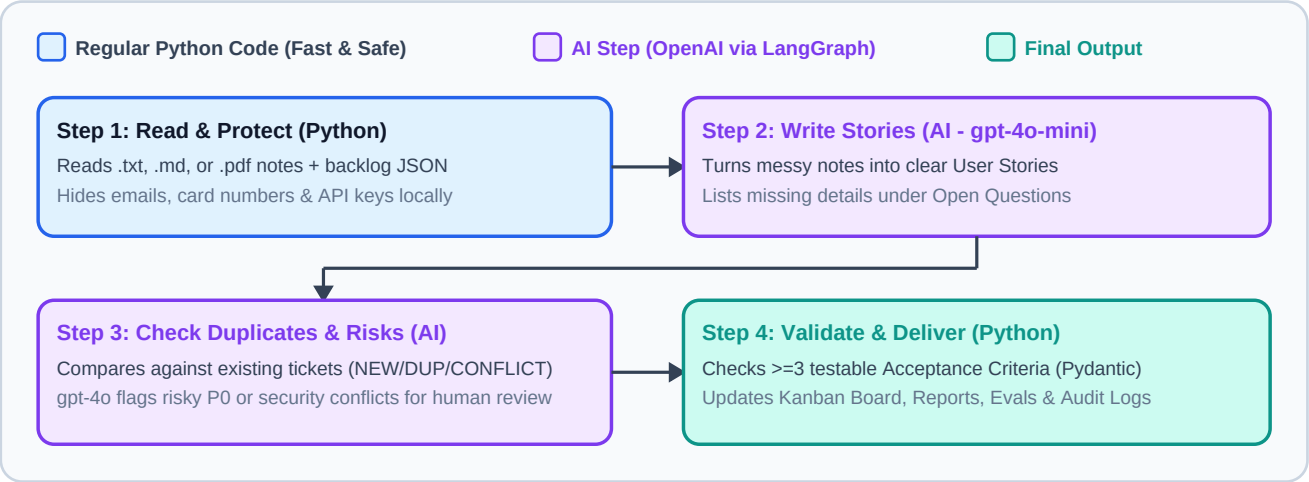
- **Duplicate Tickets:** Creating a brand-new ticket for work that is already in the backlog.
- **Missed Updates:** Forgetting to link new requirements to the existing ticket they change.
- **Hidden Conflicts:** Missing when a new security rule breaks an existing feature (for example, requiring encryption when a current ticket uses plain caching).
- **Made-Up Numbers:** AI guessing deadlines or SLAs when the notes didn't mention them.

Three Real-World Examples Included

Scenario	Input File	What the Assistant Produces
1. Sprint Planning	01_sprint_planning_notes.txt	Creates a new SSO login story, spots that CSV Export already exists (FIN-101), and updates Webhook Retries (FIN-104).
2. Outage Postmortem	02_incident_postmortem.md	Turns an outage report into a new circuit-breaker story and tightens timeout limits on an existing ticket (FIN-105).
3. Security PDF	03_pci_compliance_spec.pdf	Reads a PDF spec, catches a security conflict with FIN-106 (requiring human approval), and creates an audit-log story.

How AI Helped Define the Problem: Used AI pair-programming to design realistic test scenarios that cover duplicates, updates, security conflicts, and missing specifications—not just simple text summaries.

2. How the Solution Is Designed (Step 2)



How We Designed the AI Prompts (3 Iterations)

- **v1 — Simple Request:** Just asked AI for JSON stories. *Problem:* It marked everything High Priority, wrote vague criteria ('system should be fast'), and invented SLAs.
- **v2 — Role & Scoring Rules:** Gave the AI an Engineer persona, clear P0–P3 priority rules, and Fibonacci story points (1, 2, 3, 5, 8, 13).
- **v3 — Backlog-Aware & Configurable (prompts.py):** Fed the existing backlog into the prompt so the AI classifies every item as **NEW**, **UPDATES_EXISTING**, **DUPLICATE**, or **CONFLICT**, and puts any missing info into **Open Questions** instead of guessing.

How AI Helped in Design: Used AI pair-programming to structure the strict output schemas (Pydantic v2), build a 12-case Evaluation Lab with an AI Judge, and route simple tasks to **gpt-4o-mini** while saving **gpt-4o** for security checks (cutting API cost by ~94%).

3. How to Initialize & Run the Project

Follow these steps in Windows PowerShell from the project root folder. No paid API key is required to test—if OPENAI_API_KEY is not set, the app automatically runs in **Offline Mode**.

Step A — Activate Environment & Install Dependencies

```
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -e .
```

Step B — (Optional) Configure OpenAI API Key for Live AI Mode

Copy **.env.example** to **.env** and add your key (skip this step to run in free Offline Mode):

```
Copy-Item .env.example .env
# Then edit .env and set: OPENAI_API_KEY=sk-...
```

Step C — Launch the 8-Tab Interactive Web Portal

```
python -m streamlit run src/backlog_assistant/app.py --server.port 8501
```

Open **http://localhost:8501** in your browser. Use Tab 1 to chat or upload files, Tab 2 for the Kanban Board, Tab 5 for the 12-case Evaluation Lab, and Tab 8 to switch or create isolated projects.

Step D — Run CLI Analysis & Automated Tests

```
python -m backlog_assistant.cli analyze data/samples/01_sprint_planning_notes.txt --offline
python -m backlog_assistant.cli evaluate --offline
```

```
python -m pytest -v
```